

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Studiul și implementarea soluțiilor cloud scalabile
pentru aplicații web**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și tehnologia informației*
programul de studii *Ingineria informației***

Conducător științific
Ș.L.Dr.Ing. George Valentin STOICA

Absolvent
Ionescu Maria Cătălina

Anul 2025

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2025.

Absolvent: Ionescu Maria Cătălina

.....

Cuprins

Lista figurilor	iii
Lista tabelelor	iv
Lista acronimelor	v
1. Introducere	1
1.1. Prezentarea proiectului	1
1.2. Scopul proiectului	1
1.3. Structura lucrării	2
2. Aspectele teoretice ale proiectului	3
2.1. Aplicația web	3
2.2. Protocolul HTTP	3
2.2.1. Componentele sistemelor bazate pe HTTP	4
2.2.1.1. Clientul: user-agent-ul	4
2.2.1.2. Server-ul web	4
2.2.2. Conexiunea HTTP	6
2.2.2.1. Cum funcționează o conexiune HTTP	7
2.2.3. Mesaje HTTP	8
2.2.3.1. Structura mesajelor HTTP	8
2.2.3.2. Cereri HTTP	10
2.2.3.3. Răspunsuri HTTP	14
2.3. Framework-ul Flask Python	16
2.4. Bază de date distribuită MySQL	17
3. Arhitectura și Structura Aplicației	19
3.1. Introducere: Filozofia Arhitecturală și Stiva Tehnologică	19
3.2. Fluxul 1 (End-to-End): Înregistrare și Autentificare Utilizator	21
3.2.1. Interfața Frontend și Principii de Design (UI/UX)	21
3.2.1.1. Formularul de Înregistrare: Primul Contact cu Platforma	22
3.2.1.2. Formularul de Autentificare: Calea de Acces Rapid	22
3.2.2. Procesarea Backend și Mecanismele de Securitate	23
3.2.2.1. Mecanismul 1: Validarea Datelor și Prevenirea Duplicatelor	23
3.2.2.2. Mecanismul 2: Securizarea Criptografică a Parolelor (bcrypt)	23
3.2.2.3. Mecanismul 3: Gestiunea Stării prin Sesiuni Securizate	24
3.2.3. Procesarea Backend și Mecanismele de Afișare	25
3.2.3.1. Mecanismul de Căutare și Filtrare Avansată	25
3.2.3.2. Motorul de Recomandări și Randarea Rezultatelor	26
3.2.4. Ruta (pagina principală)	26
Bibliografie	28

Anexa A. Rezolvare ”Cinci Pâini”	30
Anexa B. Alte povești și basme	31

Lista figurilor

2.1. Structura user-agent-ului	4
2.2. Componentele URL-ului	4
2.3. Tipul de protocol URL	5
2.4. Numele resursei și portul	5
2.5. Calea către resursă	6
2.6. Parametrii	6
2.7. Ancoră URL	6
2.8. Diagrama modului de funcționare a server-ului web	7
2.9. Diagrama conexiunii HTTP	7
2.10. Cerere HTTP	8
2.11. Răspuns HTTP	8
2.12. Răspuns HTTP	9
2.13. Exemplu de cerere HTTP	10
2.14. Structura cererii HTTP	10
2.15. Structura unei linii de cerere HTTP	10
2.16. Forma de origine	11
2.17. Forma absolută	12
2.18. Forma de autoritate	12
2.19. Forma asterisk (*)	12
2.20. Header request	12
2.21. Date din corp sub formă de key=values	13
2.22. Date din corp împărțite în mai multe componente	13
2.23. Exemplu de răspuns HTTP în care un nou utilizator este creat	14
2.24. Structura răspunsului HTTP	14
2.25. Status line - alcătuire	15
2.26. Exemplu de response și representation header	15
3.1. Diagrama arhitecturală a componentelor sistemului distribuit	20
3.2. Structura ierarhică a proiectului Flask	20
3.3. Biblioteci principale utilizate în proiect	20
3.4. Interfața formularului de înregistrare, proiectată pentru simplitate și claritate.	22
3.5. Interfața formularului de autentificare, focalizată pe funcționalitate.	23
3.6. Crearea și utilizarea unei sesiuni de utilizator în Flask.	25
3.7. Interfața formularului de autentificare, focalizată pe funcționalitate.	25
3.8. Interfața formularului de autentificare, focalizată pe funcționalitate.	25
3.9. Exemplu de pagină cu rezultatele căutării, afișând proprietățile disponibile.	26

Lista tabelelor

B.1. Alte basme.	31
--------------------------	----

Lista acronimelor

HTML = HyperText Markup Language

HTTP = Hypertext Transfer Protocol

IoT = Internet of Things

VM1 = Mașină virtuală 1

VM2 = Mașină virtuală 2

Capitolul 1

Introducere

1.1 Prezentarea proiectului

Cerințele privind performanța aplicațiilor web sunt în continuă creștere datorită gamei extinse de dispozitive inteligente și IoT (Internet of Things) folosite de către utilizatori pentru a-și îmbunătăți viața de zi cu zi. Numărul mare de dispozitive care accesează în mod concomitent un site web duce la un volum copleșitor de cereri către servere pentru a accesa informația cerută, ceea ce poate cauza o experiență întârziată sau chiar imposibilă a utilizatorului. Datorită acestor circumstanțe, traficul de date crește, iar aplicațiile web trebuie să facă față la un flux considerabil de informații. La momentul actual, o soluție pentru optimizarea aplicațiilor web expuse unui trafic mare de date, dar și pentru a spori satisfacția utilizatorilor este folosirea soluțiilor scalabile a resurselor software și hardware.

Scalabilitatea [1] reprezintă capacitatea unui sistem de a-și folosi în mod adaptiv și utiliza eficient resursele în scopul creșterii performanței și reducerii costurilor, în funcție de cerințele de procesare. Aceasta se clasifică după tipul de resurse folosite în:

1. Scalabilitatea resurselor software [2]: se referă la abilitatea unei aplicații de a-și adapta sau menține performanța atunci când este o creștere sau o scădere bruscă a volumului de utilizatori, date sau operațiuni.
2. Scalabilitatea resurselor hardware [3]: reprezintă capacitatea unui sistem de a-și mări capacitatea sau de a-și crește performanța utilizând noi resurse hardware.

De asemenea, folosirea în paralel a unei bazei de date distribuită reprezintă o soluție eficientă pentru asigurarea redundanței datelor și a performanței optime într-o aplicație complexă.

O bază de date distribuită [4] este o bază de date controlată de un sistem de gestiune a bazelor de date (Data Base Management System), în care datele sunt distribuite la mai multe calculatoare interconectate.

1.2 Scopul proiectului

Proiectul „**Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web**” își propune studierea, implementarea și testarea soluțiilor scalabile software pentru o aplicație web, utilizând framework-ul Flask Python (pentru dezvoltarea aplicației web), MySQL (sistem de gestionare a bazelor de date relaționale) și Nginx (loadbalancer și server web de tip reverse proxy).

În mod ideal, scalabilitatea presupune gestionarea optimă a resurselor hardware și software, astfel încât atunci când aplicația este expusă asupra unui număr mare de cereri, ea va trebui să le gestioneze eficient, fără a compromite viteza de răspuns sau epuiza toate resursele disponibile. Scalabilitatea software [5] reprezintă implementarea unor soluțiilor de tip software care asigură funcționarea eficientă a aplicației la un volum de utilizatori brusc variabil. Aceasta folosită concomitent cu scalabilitatea hardware, ce are rol în extinderea resurselor fizice ale

unui sistem, poate aduce la rezultate impresionante în gestionarea traficului și a numărului de cereri semnificativ pentru a oferi utilizatorilor o experiență cât mai fluidă.

În practică, oricât de mult am încerca să avem o viteză de răspuns cât mai optimă, aceasta implică un cost foarte mare al resurselor software, hardware dar și a energiei electrice folosite. Un volum limitat de resurse hardware și software implică, de asemenea, o eficiență de gestionare a cererilor din cadrul unei aplicații web mai scăzute. Astfel, este necesară o arhitectură software și hardware bine definită pentru a reuși în atingerea unor rezultate promițătoare cu un număr minim de resurse folosite.

Scopul acestui proiect este a urmări și analiza capabilitatea aplicației de a răspunde la un număr mare de cereri simultane, îmbunătățirea performanței și reducerea redundanței prin implementarea unei soluții scalabile de tip software care include un load balancer și o bază de date distribuită master-slave.

Aplicația, o platformă de tip web destinată rezervărilor, a fost dezvoltată într-un mediu de lucru alcătuit dintr-o stație de lucru, alături de un mediu de testare, constituit de cele două mașini virtuale instalate pe stația de lucru. Împreună, cele trei mașini constituie o rețea IP/LAN privată. De asemenea, la nivel arhitectural s-a implementat o bază de date distribuită de tip master-slave pentru a optimiza operațiile de citire-scriere.

Obiectivele specifice acestei lucrări sunt:

- Dezvoltarea aplicației web folosind framework-ul Flask Python care include rute HTTP pentru funcționalități de bază precum ar fi: autentificarea (login, register), managementul utilizatorilor (profile), administrarea rezervărilor (reserve) și pagini administrative (admin).
- Implementarea arhitecturii scalabile prin intermediul sistemului de load balancing folosind Nginx. Nginx are rolul de a distribui cererile trimise între cele două servere disponibile, aplicația Flask și VM2, rezultând în creșterea performanței și puterii de procesare a aplicației web.
- Optimizarea performanței bazei de date cu un server master pe stația de lucru și un server slave pe VM1. Cele două servere îmbunătățesc operațiile de citire și scriere și asigură redundanța datelor prin atribuirea operațiilor de citire serverului master, iar operațiile de scriere serverului slave.

Semnificația practică a acestei lucrări constă în evidențierea rezultatelor obținute în urma aplicării soluțiilor scalabile implementate pentru a facilita o performanță crescută folosind un număr de resurse limitate în gestionarea eficientă a cererilor utilizatorilor. Rezultatele oferă o perspectivă clară asupra comportamentului unei aplicații web atunci când este supusă la un volum semnificativ de cereri, dar și asupra faptului cum pot fi îmbunătățite performanța și costurile de operare .

1.3 Structura lucrării

Capitolele următoare vor prezenta principalele concepte și soluții abordate în cadrul acestei lucrări, cum ar fi: aspectele teoretice ale proiectului, tehnologiile folosite în implementare și structura aplicației. Acestea sunt urmate de descrierea arhitecturii, în care sunt evidențiate diferitele module ce constituie aplicația și interacțiunea dintre ele la nivel scalabil. Capitolele vor cuprinde și exemplificări cu fragmente de cod care au un rol crucial în aplicarea fenomenului de scalabilitate asupra aplicației web. De asemenea, lucrarea abordează explicații amănunțite despre modul de funcționare și utilizare a interfeței grafice a aplicației cu ajutorul capturilor de ecran.

Capitolul 2

Aspectele teoretice ale proiectului

2.1 Aplicația web

O aplicație web [6] este o aplicație de tip software accesată prin intermediul unui browser web care poate rula pe un server local sau cloud. Aceasta poate fi accesată de pe orice tip de dispozitiv conectat la internet cum ar fi calculatoarele, telefoanele, consolele ș.a.m.d. și este implementată prin intermediul unor limbaje de programare web ca HTML, CSS și JavaScript. Ceea ce deosebește o aplicație web față de cele mobile sau desktop este faptul că poate să ruleze pe unul sau pe mai multe servere și este nevoie doar de un browser web pentru a o accesa.

Principalele caracteristici și funcționalități ale unei aplicații web sunt:

- Scalabilitatea: datorită faptului că aplicațiile web rulează pe un server există posibilitatea de a implementa soluții scalabile care să îmbunătățească și optimizeze performanța pentru a răspunde la cerințele utilizatorilor.
- Accesibilitatea: oricine are un dispozitiv cu conexiune la internet poate să acceseze aplicațiile web, indiferent de locație sau specificațiile hardware și software pe care le are dispozitivul folosit.
- Integrarea cu alte servicii și aplicații: aplicații web fiind versatile, programatorii au posibilitatea de a rafina experiența utilizatorilor cu ajutorul serviciilor și aplicațiilor web suplimentare concepute pentru a personaliza experiența fiecărei persoane.
- Actualizarea eficientă: odată ce s-a confirmat de către echipa de testare faptul că noua versiune a aplicației web a trecut toate testele și îndeplinește toate cerințele, aceasta poate fi lansată către public fără a fi necesară intervenția din partea lui, actualizarea fiind automată pe server.

2.2 Protocolul HTTP

În contextul vremurilor actuale, aplicațiile web au devenit unul dintre cele mai rapide, populare, dar și accesibile metode de a accesa informații și servicii de către utilizatori, doar la un click distanță. La baza acestora se află o arhitectură de tip client-server, unde interacțiunile sunt realizate prin intermediul protoalelor de comunicație, cel mai des utilizat fiind protocolul HTTP.

Protocolul HTTP (Hypertext Transfer Protocol) [7] este un protocol la nivelul stratului de aplicație, fără reținerea stării (serverul nu reține date despre sesiunea activă), de tip client-server care stă la baza oricărui schimb de date între un client și un server pentru preluarea unor resurse disponibile pe internet, cum ar fi documentele HTML (HyperText Markup Language), fișiere audio, video ș.a.m.d.

Comunicarea dintre clienți și servere are loc prin schimbul de mesaje individuale, în care mesajele trimise de client poartă numele de cereri, iar mesajele trimise de către server - răspunsuri.

Un client începe o conexiune pentru a face o cerere, așteaptă până primește un răspuns de la server, iar serverul în cele din urmă înapoiază informațiile solicitate de către client.

2.2.1 Componentele sistemelor bazate pe HTTP

2.2.1.1 Clientul: user-agent-ul

User-agent-ul [8] este un header de tip string folosit într-o cerere HTTP pentru a trimite informații despre aplicație, sistemul de operare, furnizorul și/sau versiunea folosită către server.

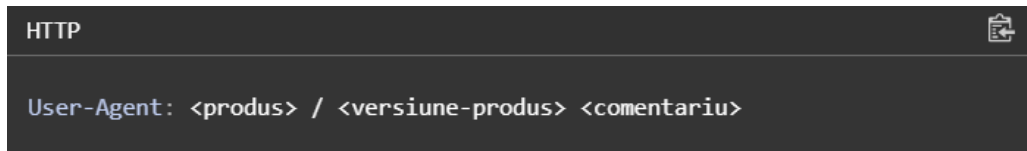


Figura 2.1: Structura user-agent-ului

Ca să afișeze o pagină web, în primă fază, browser-ul trimite o cerere inițială pentru a prelua documentul HTML solicitat de către client. În cea de-a doua fază, acesta analizează fișierul primit și trimite cereri suplimentare pentru a obține informații despre stilizarea paginii (CSS), eventuale scripturi ce trebuie executate și sub-resursele aflate în pagină (imagini sau videoclipuri). În fază finală a procesului, browser-ul combină toate aceste resurse pentru a afișa varinta finală a documentului.

2.2.1.2 Server-ul web

Server-ul web [9] se ocupă cu livrarea resurselor cerute de către client. Din punct de vedere al structurii hardware, acesta poate fi reprezentat de către o stație de lucru individuală sau de o colecție de servere interconectate în aceeași rețea care au rol în a gestiona schimbul de date stocate (de exemplu: documente HTML, imaginii, fișiere JavaScript) cu alte dispozitive.

Pe de altă parte, din punct de vedere al structurii software, un server web are rolul de a interpreta adresele URL și de a procesa cererile și răspunsurile transmise prin protocolul HTTP.

Componente URL

URL (Uniform Resource Locators) [10] reprezintă adresa unei resurse unice de pe internet și unul dintre mecanismele folosite de browsere pentru a prelua resurse cum ar fi pagini HTML, documente, imagini ș.a.m.d. Orice URL introdus în bara de adrese trimite o solicitare directă către un server web pentru a accesa resursa specifică, iar în funcție de răspunsul serverului, resursa solicitată fie este afișată, fie un mesaj de eroare este returnat. Conform figurii 2.8, un URL este alcătuit din mai multe componente cum ar fi: tipul de protocolul URL, numele resursei, portul, calea către resursă, parametrii și fragmentul de identificare internă.

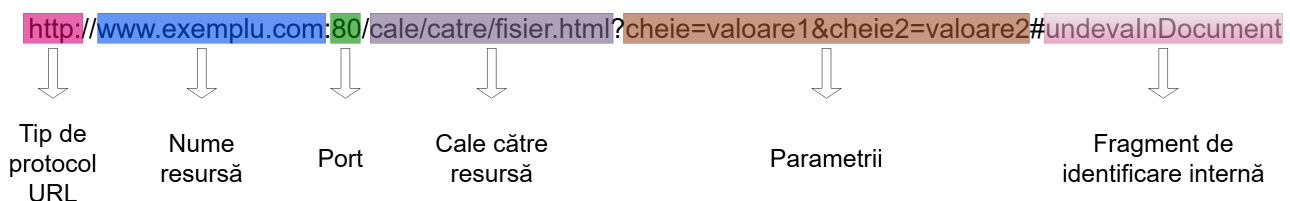


Figura 2.2: Componentele URL-ului

Tipul de protocol URL

Prima componentă regăsită într-un URL reprezintă tipul de protocol pe care browser-ul trebuie să îl folosească pentru a trimite o cerere de acces la resursă. Protocelele cele mai des întâlnite sunt HTTPS (Hypertext Transfer Protocol Secured) sau HTTP (variante nesecurizată), dar există cazuri în care este folosit și protocolul mailto: care deschide un client de tip mail.



Figura 2.3: Tipul de protocol URL

Numele resursei și portul

Cea de-a doua componentă a URL-ului este formată din numele resursei și port. Numele resursei sau numele domeniului, separat de tipul de protocol prin șirul de caractere `://`, reprezintă adresa paginii web redactată sub forma unui șir de caractere. Orice pagină web are atribuit un IP public unic din intervalul 1.0.0.0 - 9.255.255.255, numere care sunt greu de reținut sau scris pentru orice persoană. Din acest motiv, cu ajutorul numele domeniului, procesul de accesare a unei pagini web devine mai rapid pentru utilizatori prin „traducerea în limbaj natural” a adreselor IP într-un format ușor de înțeles și accesibil pentru oricine. Portul este o „poartă” tehnică folosită pentru a accesa resursele pe serverul web. În cele mai multe cazuri, declararea portului este omisă deoarece majoritatea serverelor folosesc în mod implicit portul 80 pentru conexiunea HTTP și 443 pentru conexiunea HTTPS, dar în situațiile în care este necesară utilizarea unui alt port, acesta trebuie declarat în mod obligatoriu.

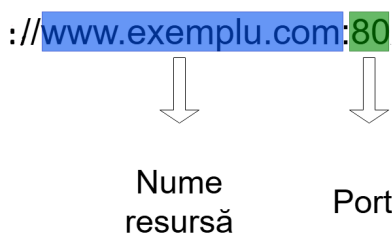


Figura 2.4: Numele resursei și portul

Calea către resursă

Următoarea componentă a unui URL este calea către resursă. Calea indică locația exactă a unei pagini, imagini sau altei resurse aflate pe server.

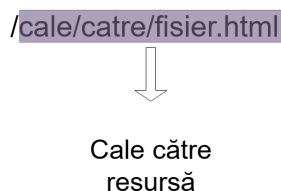


Figura 2.5: Calea către resursă

Parametrii

Cea de-a patra componentă constă într-un atribut unic, denumit parametrii. Parametrii sunt o listă de chei și valori folosite, delimitate de simbolul & de către serverul web utilizat pentru transmiterea de informații suplimentare către server. Fiecare server web are o listă de chei sau de valori specifice, nu există o regulă specifică în crearea acestora.

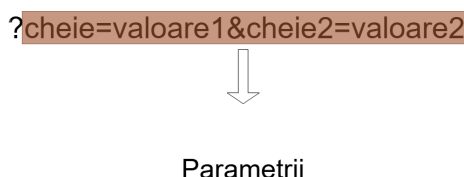


Figura 2.6: Parametrii

Fragment de identificare internă

Ultima componentă ce intră în alcătuirea unui URL se numește fragment de identificare internă. Un fragment de identificare internă sau ancoră, delimitată de simbolul #, are rol în afișarea unei anumite secțiuni din resursă. Ancorele funcționează ca un fel de „semn de carte”, acestea indicând aplicației web până unde va trebui să parcurgă documentul ca să ajungă la secțiunea marcată și într-un final să o afișeze.

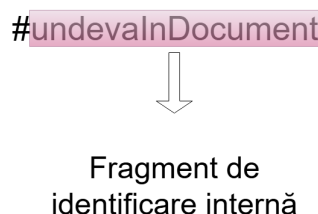


Figura 2.7: Ancoră URL

2.2.2 Conexiunea HTTP

Accesarea directă a unui server HTTP se realizează prin introducerea URL-ului în bara de adrese, acesta livrând conținutul solicitat utilizatorilor. La nivel fundamental, modul de funcționare presupune ca de fiecare dată când un browser are nevoie de un fișier stocat pe un server web, acesta va iniția o cerere HTTP pentru a primi fișierul respectiv. Odată ce serverul HTTP a recepționat cererea, verifică dacă URL-ul solicitat există. În situația în care resursa este disponibilă, serverul trimite utilizatorilor conținutul corespunzător. În cazul contrar, serverul mai întâi acesta verifică dacă este necesară generarea unui fișier dinamic pentru a răspunde solicitării. În cele din urmă, dacă niciuna dintre opțiuni nu este disponibilă, serverul va returna codul de eroare 404 Not Found.

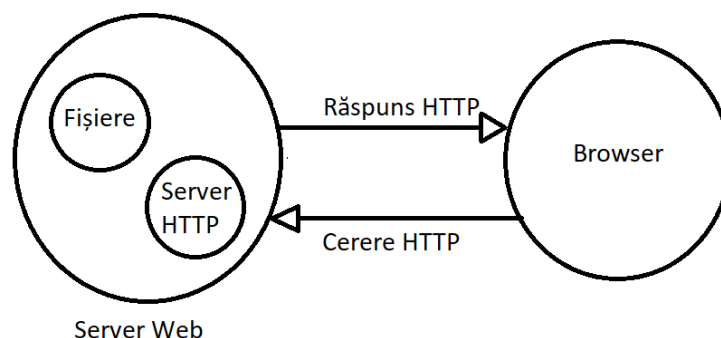


Figura 2.8: Diagrama modului de funcționare a server-ului web

2.2.2.1 Cum funcționează o conexiune HTTP

O conexiune HTTP este controlată la nivelul stratului de transport, ceea ce implică faptul că se va folosi un protocol de transport pentru gestionarea conexiunii între client și server. Pentru a transmite mesaje cu o rată de pierdere foarte mică s-a optat în folosirea protocolului TCP. Datorită fiabilității sale, se utilizează protocolul TCP care garantează trimiterea mesajelor cu un număr minim erori, cu prețul unei viteze de transfer mai mici.

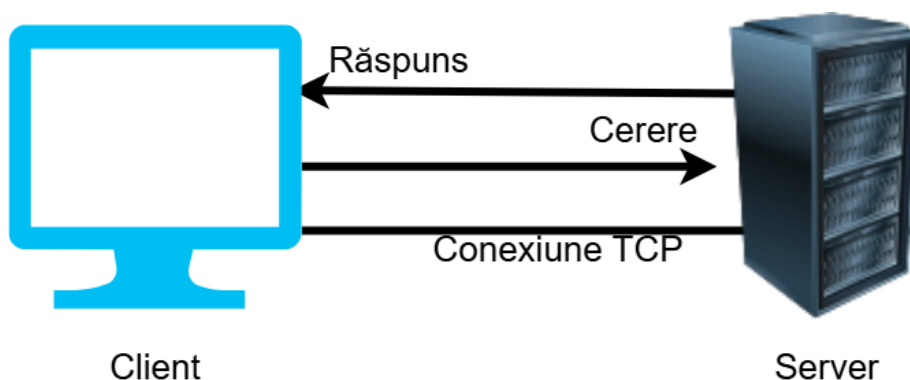


Figura 2.9: Diagrama conexiunii HTTP

Primul pas spre a începe o conexiune HTTP îl reprezintă schimbul de cerere/răspuns dintre client și server. Clientul are posibilitatea de a deschide o nouă conexiune, de a folosi o conexiune existentă sau de a deschide mai multe conexiuni TCP către servere. Acest schimb începe odată ce s-a stabilit o conexiune TCP, un proces care trimite unul sau mai multe cereri din partea clientului pentru a primi un răspuns de la server.

Al doilea pas presupune trimiterea mesajului HTTP către server și răspunsul serverului la cererea clientului cu informația cerută. Dacă serverul răspunde cu codul 200 OK, înseamnă că s-a stabilit cu succes conexiunea, iar serverul poate transmite clientului informația cerută.

```

HTTP

GET / HTTP/1.1
Host: localhost
Accept-Language: en

```

Figura 2.10: Cerere HTTP

```

HTTP

HTTP/1.1 200 OK
Date: Sat, 09 Oct 2025 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!doctype html>... (conținutul paginii cerute)

```

Figura 2.11: Răspuns HTTP

În cele din urmă, odată ce s-a făcut schimbul de resurse, există posibilitatea de a închide sau reutiliza conexiunea pentru viitoare sesiuni.

2.2.3 Mesaje HTTP

Mesajele HTTP sunt șiruri de caractere scrise în limbaj natural, simplist, începând cu versiunea HTTP/1.1 până la versiunea HTTP/2. Acestea sunt clasificate în două categorii: cerere (request) și răspuns (response), reprezentând un mecanism de schimb de date între server și un client. O cerere este trimisă de către client serverului-ui cu scopul în a declanșa o acțiune, iar răspunsul are rolul de a furniza informația solicită înapoi către client. Crearea și modificarea mesajelor HTTP se datorează API-urilor disponibile în browser, fișierelor de configurație ale proxy-urilor sau serverelor sau altor interfețe software dedicate.

2.2.3.1 Structura mesajelor HTTP

Conform figurei 2.12 se poate observa faptul că mesajele de tip cerere și răspuns prezintă o structură asemănătoare. Partea superioară (head) a mesajului HTTP este alcătuită din linia de start și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională.

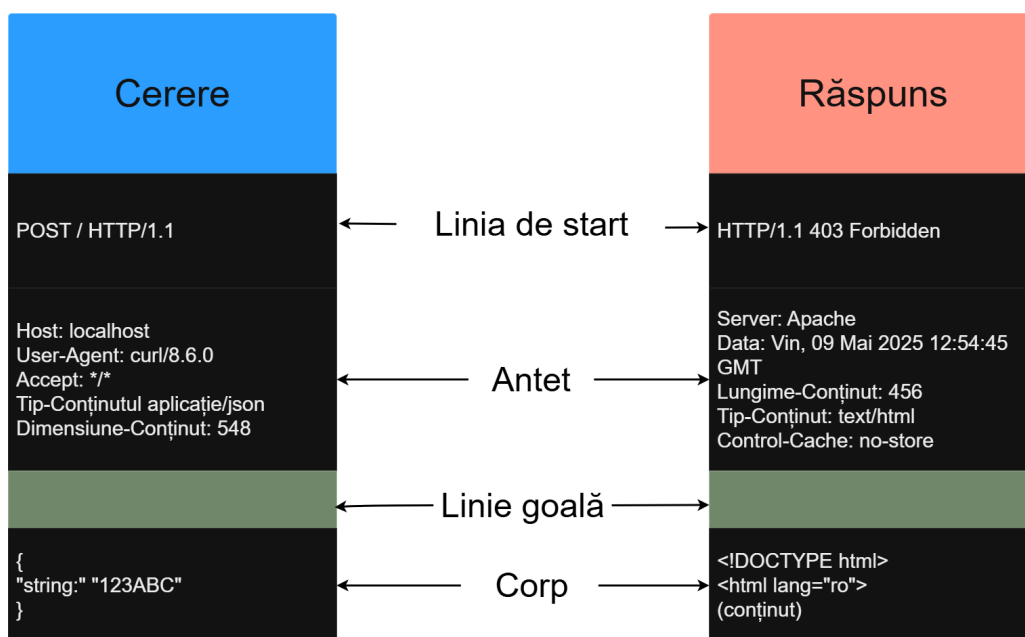


Figura 2.12: Răspuns HTTP

Partea superioară conține informații esențiale pentru interpretarea mesajului cum ar fi:

1. Linia de start, compusă dintr-un singur rând, precizează versiunea HTTP, precum și metoda cererii sau codul de răspuns.
2. Antetul (header) conține metadata folosite pentru a descrie mesajul transmis și facilitează schimbul de informații suplimentare dintre un client și un server, fie în cadrul unei cereri, fie al unui răspuns. În funcție de rolul pe care îl îndeplinesc, se remarcă mai multe categorii de antet:
 - Antet de cerere (request header): conține informații despre resursa ce urmează a fi preluată.
 - Antet de răspuns (response header): oferă informații despre răspuns, precum ar fi locația sau serverul de pe care provine.
 - Antet de reprezentare (representation header): cuprinde informații despre corpul resursei, precum ar fi tipul de conținut sau de compresie sau codare.
 - Antet cu conținut util (payload header): include informații referitoare la datele din corpul mesajului (payload) ce descriu caracteristici ale mesajului, fără a depinde de modul în care acesta este procesat sau trimis .
3. O linie goală care indică faptul că metadata mesajului este complet.

Un corp (body) pune la dispoziție datele asociate mesajului, transmiterea căruia este stabilită încă de la linia de start și din anteturile anterioare. De obicei corpul include fie o metodă trimisă de client către server, fie o resursă redirectionată de către server clientului.

2.2.3.2 Cereri HTTP

```

HTTP

POST /user HTTP/1.1
Host: exemplu.ro
Content-Type: application/x-www-form-urlencoded
Content-Length: 78

nume=nume%20&prenume&email=licenta%40exemplu.ro

```

Figura 2.13: Exemplu de cerere HTTP

O cerere HTTP [11] este un mesaj trimis de client către server cu scop de a solicita o resursă. Structura acesteia este alcătuită dintr-o parte superioară ce cuprinde linia cererii (request line), antet (header), urmată de o linie goală și opțional, un corp.

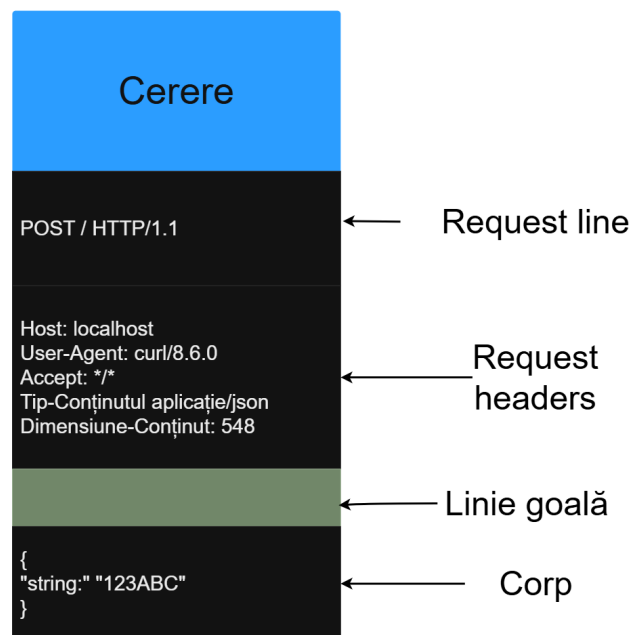


Figura 2.14: Structura cererii HTTP

Structura părții superioare a unei cereri HTTP este următoarea:

1. Linia cererii: este alcătuită din trei elemente precum ar fi metoda, resursa solicitată și protocolul.

```

HTTP

<metodă> <resursa-solicitată> <protocol>

```

Figura 2.15: Structura unei linii de cerere HTTP

Metoda HTTP

O metodă HTTP indică scopul unei cererii trimise către un server și rezultatul așteptat în cazul în care cererea a fost realizată cu succes. De obicei fiecare metodă are propriul său rol, dar pot împărtăși anumite caracteristici comune. Există mai multe tipuri de metode precum ar fi:

- **GET** este folosită pentru doar a prelua date de pe server, fără ca acestea să sufere noi modificări.
- **HEAD** se comportă în același mod ca și metoda GET, dar fără a include conținut în răspuns.
- **POST** are rol în a trimite date către server, de exemplu încărcarea unui fișier.
- **PUT** înlocuiește toate instanțele curente cu noi instanțe specificate de către client.
- **DELETE** este utilizată cu scopul de a solicita ștergerea unei resurse specificate.
- **CONNECT** crează un tunel de comunicație cu serverul identificat prin resursa țintă.
- **OPTIONS** are rol în a descrie opțiunile de comunicare posibile pentru resursa specificată.
- **TRACE** este folosită pentru efectuarea unui test de buclă (loop-back) a mesajului de-a lungul traseului către resursa specificată.
- **PATCH** aplică modificări parțiale unei resurse.

Resursa solicitată

Resursa solicitată este un URI (Uniform Resource Identifier) care identifică resursa căreia i se trimite cererea. Există mai multe tipuri de formatare utilizate pentru a descrie o resursă solicitată:

- Forma de origine, utilizată cu metodele GET, POST, HEAD și OPTIONS, este cea mai folosită formatare pentru a identifica originea resursei dintr-un server sau gateway. Presupune combinarea unei căi absolute cu informațiile aflate în antetul „Host” la care există posibilitatea adăugării unei interogări de forma *cheie = valoare* pentru a transmite informații suplimentare.

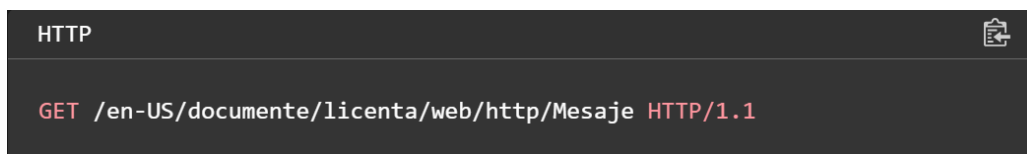
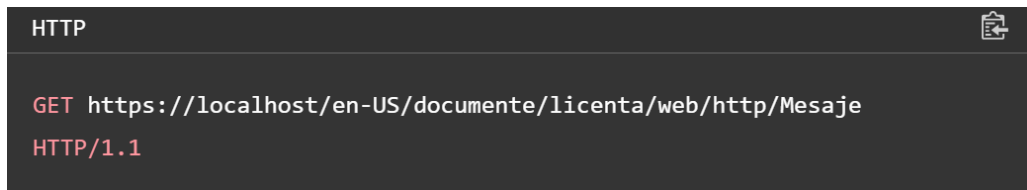


Figura 2.16: Forma de origine

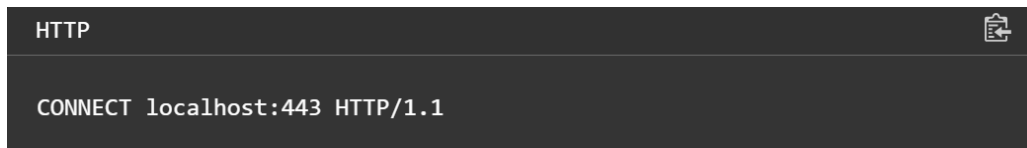
- Forma absolută este o sintaxă de tip URL complet ce include informațiile necesare pentru identificarea serverului. Atunci când cererea HTTP este făcută către un proxy, forma absolută are rol în redirectionarea răspunsului sau serviciul de la un cache valid înapoi către client, utilizând metoda GET.



```
HTTP
GET https://localhost/en-US/documente/licenta/web/http/Mesaje
HTTP/1.1
```

Figura 2.17: Forma absolută

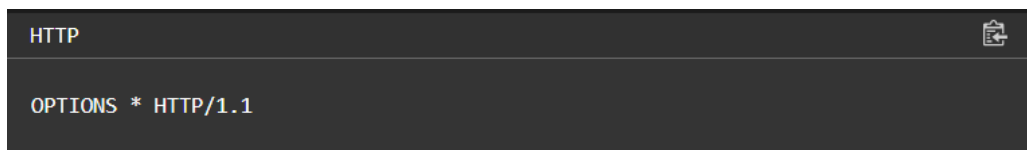
- Forma de autoritate este o structură folosită în cererile HTTP formată dintr-un nume de gazdă și opțional un port delimitat de simbolul : împreună cu metoda CONNECTION pentru a seta un tunel către resursa țintă.



```
HTTP
CONNECT localhost:443 HTTP/1.1
```

Figura 2.18: Forma de autoritate

- Forma asterisk (*) este folosită alături de metoda OPTIONS atunci când cererea HTTP nu se aplică unei resurse particulare, ci serverul-ui ca un întreg.



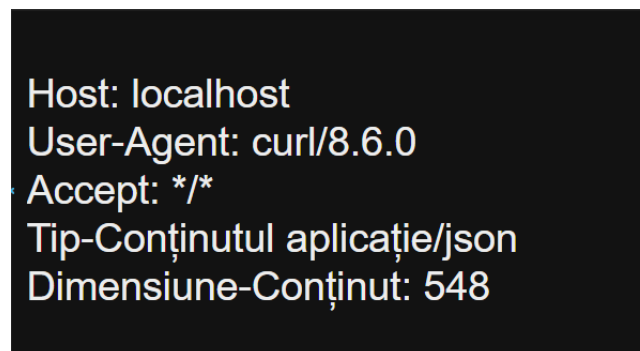
```
HTTP
OPTIONS * HTTP/1.1
```

Figura 2.19: Forma asterisk (*)

Protocol

Protocolul este un indicator care reprezintă versiunea protocolului HTTP utilizată în timpul procesului de trimitere a cererii folosit având rol în interpretarea corectă a mesajelor transmise. Singura versiune care trebuie declarată în acest câmp este HTTP/1.1, deoarece versiunile HTTP/0.9, HTTP/1.0 sunt învechite și nu se mai folosesc, iar versiunile mai noi de HTTP/2.0 sunt configurate în timpul conexiunii.

2. Antetul (header) conține metadata despre mesajul ce urmează a fi transmis și se distinge în mai multe tipuri:

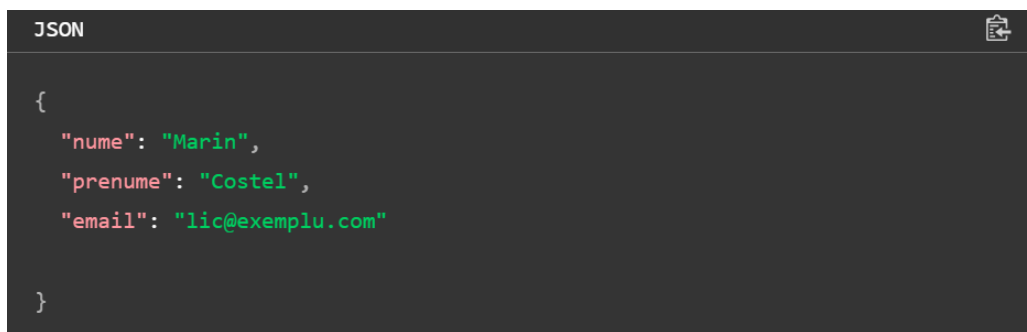


```
Host: localhost
User-Agent: curl/8.6.0
Accept: */*
Tip-Conținutul aplicație/json
Dimensiune-Conținut: 548
```

Figura 2.20: Header request

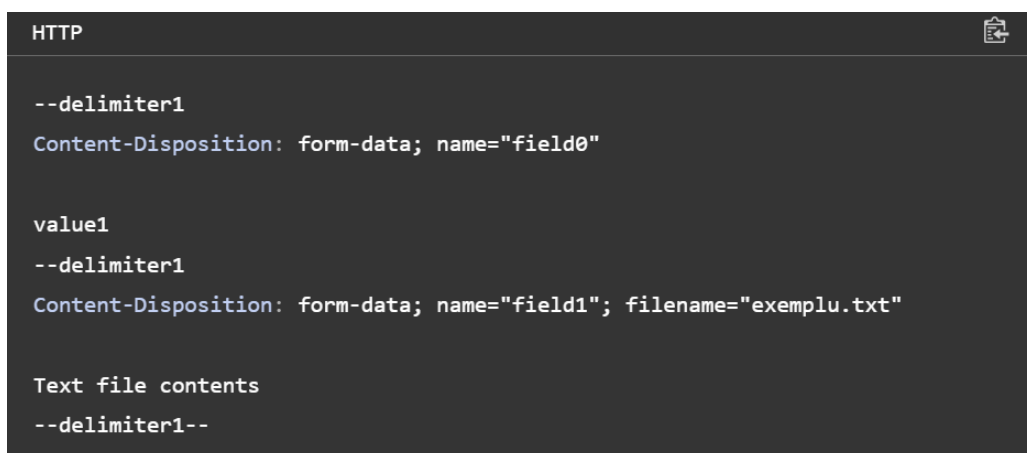
- Antet de cerere (request header) furnizează context suplimentar în cadrul unei cereri sau adaugă instrucțiuni despre cum ar trebui să fie un mesaj tratat de către server
- Antet de reprezentare (representation header) este folosit numai în cazul în care cererea prezintă și o parte superioară. Un representation request îi permite serverului să reconstruiască resursa așa cum era înainte de a fi transmisă în rețea și să precizeze forma de codare aplicată asupra mesajului. Câteva exemple des folosite într-o cerere sunt:
 - Accept-Charset
 - Host
 - User-Agent
 - Accept-Language
 - Accept-Encoding
 - Connection
 - Referer

Ultima componentă a unei cereri, compatibilă doar cu metodele PATCH, POST și PUT, este constituită din corpul mesajului care are rol în transmiterea informației către server. Corpul unui mesaj poate conține o cantitate scăzută de date sub formă de perechi de tip key=value (cheie=valoare) (2.21) sau date în mai multe componente (2.22):


 A screenshot of a code editor window titled "JSON". The editor shows a valid JSON object with three key-value pairs: "nume" with value "Marin", "prenume" with value "Costel", and "email" with value "lic@exemplu.com". The code is color-coded: keys are in red, values are in green, and string literals are in blue.


```
{
  "nume": "Marin",
  "prenume": "Costel",
  "email": "lic@exemplu.com"
}
```

Figura 2.21: Date din corp sub formă de key=values


 A screenshot of a code editor window titled "HTTP". The editor shows an HTTP request body for a multipart form data submission. It includes two parts separated by "boundary" markers. The first part has a "Content-Disposition" header with "name='field0'" and a "value1" body. The second part has a "Content-Disposition" header with "name='field1'" and "filename='exemplu.txt'", followed by the text "Text file contents".


```
--delimiter1
Content-Disposition: form-data; name="field0"

value1
--delimiter1
Content-Disposition: form-data; name="field1"; filename="exemplu.txt"

Text file contents
--delimiter1--
```

Figura 2.22: Date din corp împărțite în mai multe componente

2.2.3.3 Răspunsuri HTTP

```

HTTP
-----
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://exemplu.ro/user/321

{
  "mesaj": "Utilizator nou creat",
  "user": {
    "id": 321,
    "nume": "Marin",
    "prenume": "Costel",
    "email": "lic@exemplu.com"
  }
}

```

Figura 2.23: Exemplu de răspuns HTTP în care un nou utilizator este creat

Răspunsurile HTTP [12] sunt mesaje trimise de către server către clienți, în care răspunsul anunță clientul rezultatul determinat în urma cererii. Partea superioară (head) a mesajului HTTP este alcătuită din linia de status și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională.

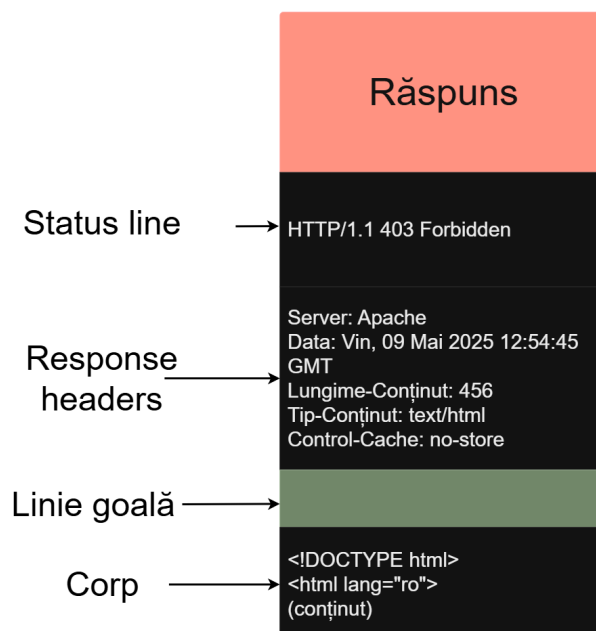


Figura 2.24: Structura răspunsului HTTP

Structura părții superioare a unui răspuns HTTP este următoarea:

1. Status line este format din trei componente:

- Protocolul reprezintă versiunea protocolului HTTP pe care serverul l-a folosit pentru a transmite mesajul către client.
- Codul de stare indică dacă starea cu care s-a finalizat transmiterea mesajului HTTP. Există cinci tipuri de coduri de stare cum ar fi: răspunsuri informaționale (cu valori

între 100 și 199) ce descriu transmiterea cu succes (200-299), mesaje de redirecționare (300-399), răspunsuri pentru erori de client (400-499) și răspunsuri pentru erori de server (500-599). Cele mai des întâlnite coduri de stare sunt 200 - OK (mesajul a fost transmis cu succes), 404 - Not Found (serverul nu poate găsi resursa cerută, URL-ul nu este recunoscut de către browser) și 302 - Found (URL-ul asociat resursei căutate a fost schimbat temporar).

- Textul de stare este un text pur informativ care explică semnificația codului de stare pentru a facilita o mai bună înțelegere a rezultatului generat în urma răspunsului.



Figura 2.25: Status line - alcătuire

2. Antetul unui răspuns conține metadate despre mesajul ce urmează a fi transmis clientului și se clasifică în:

Response header

Un response header este un antet cu metadatele trimise odată cu răspunsul de server către client. Fiecare antet este un șir de caractere care nu face diferențe între majuscule și litere mici, urmate de simbolul : și o valoare care este dependentă de tipul de antet folosit. Scopul response header-ului este de a oferi informații suplimentare despre mesaj sau de a explica logica cu care va trebui clientul să gestioneze cererile ce o să urmeze.

Representation header

Representation header-ul descrie forma datelor din mesaj, orice codare aplicată asupra acestuia și poate exista doar dacă mesajul are un corp.

Un representation header oferă unui destinatar instrucțiuni despre cum ar trebui să reconstruiască resursa înainte de a fi trimisă în rețea.

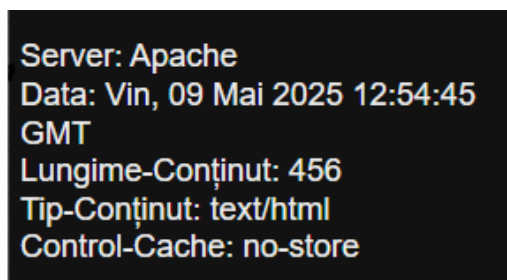


Figura 2.26: Exemplu de response și representation header

Ultima componentă a unui răspuns HTTP este corpul, prezent în majoritatea cazurilor atunci când un server trimite un răspuns către client. În cazul răspunsurilor reușite, corpul conține datele solicitate de către client prin intermediul metodei GET. În schimb, dacă apar erori în procesul de transmitere a mesajului, corpul mesajului va include detalii despre problemele care au cauzat imposibilitatea trimiterii răspunsului.

2.3 Framework-ul Flask Python

Flask [13] [14] este un framework web minimalist flexibil creat în limbajul Python, deseori numit „micro-framework” datorită accesibilității de a dezvolta aplicații web folosind doar componente esențiale, fără necesitatea folosirii unor biblioteci suplimentare sau structuri complicate și posibilitatea implementării rapide a unor noi funcționalități în aplicațiile web.

Flask a fost creat [15] în anul 2004 de către Armin Ronacher, un membru al grupului de pasionați de Python, Poccoo. Inițial, crearea celebrului framework a fost doar o aluzie la celebra zi a glumelor, 1 aprilie, dar mai târziu a devenit realitate datorită cererii mari a publicului. Numele „flask” (în engleză înseamnă flacon) este o referință la precedentul framework Bottle (în engleză sticlă). În anul 2016, echipa Poccoo și-a încheiat activitatea, iar dezvoltarea Flask-ului și a librăriilor acestuia a fost preluată de către echipa nou fondată numită Pallets.

În contextul acestei lucrări de licență, am ales să implementez serverul aplicației web în Flask datorită funcționalităților de construire personalizată a site-urilor, definirea rutelor care facilitează redirectionarea către pagini atunci când un link este accesat și gestionarea sesiunilor de autentificare a utilizatorilor, dar și a cererilor trimise de către acesta aplicației web. Printre avantajele ale framework-ului Flask se regăsesc:

- Este un framework backend minimalist cu posibilitatea extinderii aplicației în funcție de cerințele impuse.
- Este ușor de învățat deoarece API-ul intuitiv oferă posibilitatea chiar și începătorilor să se familiarizeze cu el.
- Este un framework flexibil deoarece are abilitatea de a permite personalizarea și extinderea aplicației conform cerințelor cerute.
- Poate fi folosit cu o paletă largă de baze de date precum MySQL prin intermediul extensiilor sau al bibliotecilor externe.
- Routing HTTP care se ocupă cu definirea rutelor URL și asocierea acestora cu funcții care gestionează cererile și răspunsurile HTTP.
- Sesiunile de utilizatori pot fi ușor gestionate, deoarece Flask are posibilitatea de a stoca informații despre acestea în mod securizat pe server și client utilizând cookie-uri semnate.
- Posibilă integrarea cu Jinja2 care oferă o paletă largă de unelte pentru generarea paginilor HTML dinamice.

O primă funcționalitate pe care o are Flask este gestionarea logică a aplicației web și baza de date asociată. Atunci când o acțiune de tipul citire, scriere, update sau delete are loc la nivel de frontend, Flask va fi intermediarul între utilizator și baza de date în care va traduce comanda inițiată în limbajul SQL pentru executare, în final rezultatul fiind vizibil și la nivel de frontend.

Cea de a doua funcționalitate constă în permiterea autentificării și existența sesiunii de utilizator. Flask se ocupă cu crearea noilor conturi, logarea utilizatorilor, reținerea cine este logat și ce acțiuni îi sunt permise.

O altă funcționalitate se referă la afișarea paginilor dinamice folosind template-uri HTML.

Ultma funcționalitate o reprezintă controlul fluxului aplicației în care utilizatorii după efectuarea anumitor acțiuni sunt redirectionați spre alte pagini. De exemplu, la crearea unui nou cont, utilizatorii sunt redirectionați către pagina de login unde se vor conecta la aplicație.

2.4 Bază de date distribuită MySQL

O bază de date reprezintă [16] o colecție de informații corelate organizate sub forma unui spațiu de stocare menite pentru a facilita procesele de gestionare, accesare a informațiilor de către utilizatori. Sistemul de gestiune a bazelor de date (SGBD) este un software fundamental ce stă la baza multor aplicații cum ar fi sisteme de rezervări, cumpărături online în care utilizatorii au posibilitatea să definească, să creeze, să întrețină și să controleze accesul la baza de date.

O bază de date relațională este o bază de date ce se bazează pe modelul relațional al datelor organizat sub forma unor tabele. Un tabel (sau relație) este format din rânduri și coloane. Rândurile, denumite și tupluri, reprezintă o entitate sau o înregistrare individuală în baza de date. Coloanele sau atributele, reprezintă caracteristicile sau proprietățile entităților. Fiecare coloană are un identificator unic și un tip de date specific.

O bază de date distribuită este o bază de date logică unică divizată în mai multe părți stocate fizic pe mai multe computere sau noduri interconectate în rețea. La nivel de frontend, o bază de date distribuită funcționează ca o singură bază de date centralizată, nu este nevoie ca utilizatorii să folosească mai multe interfețe, dar la nivel de backend datele sunt distribuite și replicate în diverse locații. Principalele obiective pe care o bază de date distribuită le prezintă includ scalabilitatea, disponibilitatea, fiabilitatea și performanța îmbunătățită prin alocarea unor locații cu roluri specifice.

Configurația master-slave este o structură de replicare a bazei de date în care datele de pe un master sunt copiate pe un slave. Un master este nodul principal în care au loc toate operațiile de scriere (INSERT, UPDATE, DELETE). Orice modificare efectuată asupra master-ului este transmisă și slave-lui. Slave-ul este un nod secundar ce are rol în efectuarea tuturor operațiilor de citire (SELECT) și în actualizarea datelor primite din partea master-ului.

În contextul prezentei lucrări, s-a implementat o arhitectură de tip master-slave datorită următoarelor avantaje:

- Scalabilitatea operațiilor de citire: într-un sistem de rezervări volumul operațiilor de citire depășește semnificativ volumul operațiilor de scriere. Așadar, o arhitectură de tip master-slave devine esențială prin direcționarea cererilor de citire către nodurile slave, eliberarea resurselor consumate de către nodul master și eficientizarea operațiilor de scriere importante. De asemenea, flexibilitatea sistemului asigură posibilitatea de extindere orizontală cu noi noduri slave, distribuind uniform și eficient cererile de citire pe măsură ce volumul de date crește.
- Separarea sarcinilor: arhitectura master-slave stabilește două roluri bine separate între cele două noduri ale bazei de date. Nodul master-ul se ocupă exclusiv cu procesarea și validarea operațiilor de scriere (INSERT, UPDATE, DELETE) pentru a asigura consistența și integritatea datelor. Nodurile slave au rol în preluarea și executarea operațiilor de citire (SELECT). Această separare a rolurilor optimizează performanța operațiilor de citire și scriere prin minimizarea conflictelor de blocare și contopire a resurselor.
- Îmbunătățirea disponibilității: configurația master-slave este o metodă preventivă în cazul unei defecțiuni sau indisponibilități a unei resurse. În cazul în care master-ul nu mai funcționează, slave-ul preia atribuțiile de scriere și este promovat la rolul de master. Acest mecanism se numește failover și are rol în menținerea continuității operațiilor de citire și scriere prin minimizarea timpului de nefuncționare a aplicației.

SQL (Structured Query Language) este cel mai utilizat limbaj de baze de date. Este un limbaj standardizat, declarativ, folosit în interacțiunea și gestionarea bazelor de date relațional. A fost dezvoltat de către compania IBM, prima versiune fiind concepută în anul 1970. Numele original al limbajului a fost SEQUEL (Structured English Query Language).

SQL este interfața universală pentru SGBD-ul MySQL. Se ocupă cu definirea structurii bazei de date, manipularea datelor și controlul accesului dintr-o bază de date relațională. SQL poate fi utilizat pentru a căuta informații specifice, insera noi înregistrări, a actualiza date existente și a șterge înregistrări.

Fiind un limbaj standard, SQL asigură logica de interacțiune cu baza de date care este bine definită și compatibilă cu diverse SGBD-uri. În cadrul aplicației Flask de rezervări, operațiile efectuate asupra bazei de date sunt traduse în interogări SQL, ce urmează a fi executate de către serverul MySQL.

Capitolul 3

Arhitectura și Structura Aplicației

3.1 Introducere: Filozofia Arhitecturală și Stiva Tehnologică

Creșterea exponențială a numărului de utilizatori și a volumului de date în mediul online a impus noi provocări în dezvoltarea aplicațiilor web. Sistemele software moderne, în special platformele care intermediază servicii, trebuie să gestioneze un număr ridicat de interacțiuni simultane fără a compromite performanța, securitatea și fiabilitatea. Tranziția procesului de rezervare din mediul tradițional către platforme digitale, accelerată de omniprezența dispozitivelor conectate la internet, impune constrângeri tehnice semnificative, transformând gestiunea cererilor concurente și a volumelor mari de date în provocări centrale.

În acest context, lucrarea de față propune studiul, proiectarea și implementarea unei platforme web pentru rezervări de cazare, servind drept studiu de caz pentru aplicarea unor concepte avansate de inginerie software. Obiectivul fundamental depășește crearea unei aplicații exclusiv funcționale, vizând construirea unui sistem a cărui arhitectură să fie, prin proiectare, performantă și robustă. Pentru a atinge aceste deziderate, elaborarea proiectului a fost ghidată de un set de obiective tehnice precise:

1. **Performanță:** Analiza și aplicarea tehnicilor de optimizare a timpului de răspuns, atât pentru operațiuni simple de afișare, cât și pentru procesări complexe precum căutarea și rezervarea.
2. **Modularitate:** Proiectarea sistemului conform unei paradigme modulare, în care fiecare componentă deține o responsabilitate clar definită, facilitând testarea, mentenanța și actualizarea independentă.
3. **Scalabilitate:** Implementarea unei arhitecturi capabile să susțină o creștere a numărului de utilizatori fără o degradare semnificativă a performanței, prin strategii ce permit scalabilitatea orizontală.
4. **Testabilitate:** Construirea componentelor software într-o manieră care să permită validarea funcțională prin teste unitare și de integrare, asigurând un nivel înalt de fiabilitate.

Soluția arhitecturală propusă pentru îndeplinirea acestor obiective este una distribuită, multi-nod. Sistemul nu se bazează pe o singură mașină, ci pe un ansamblu de componente interconectate, fiecare cu un rol specializat. Această structură, prezentată vizual în Figura 3.1, este fundamentul strategiei de disponibilitate și echilibrare a încărcării.

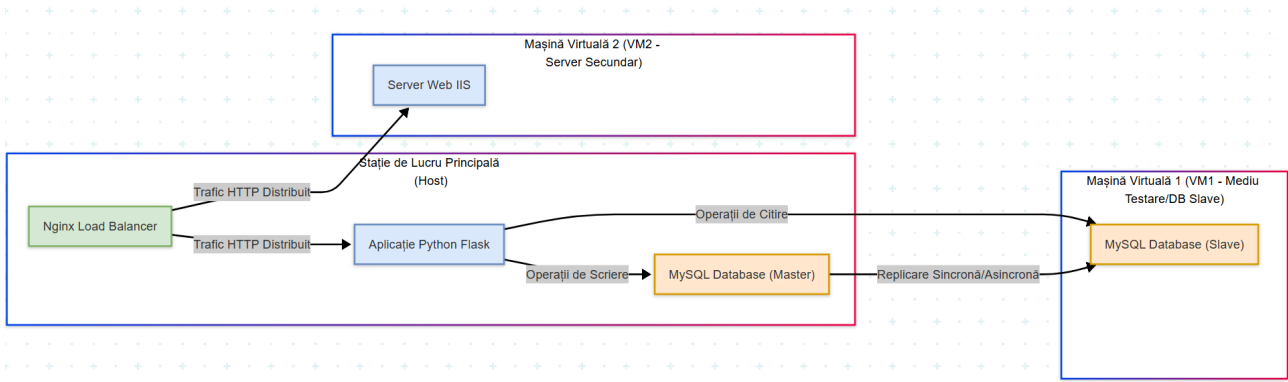


Figura 3.1: Diagrama arhitecturală a componentelor sistemului distribuit

Dacă arhitectura generală descrie interacțiunea la nivel de sistem, arhitectura internă detaliază modul în care este construită componenta centrală a acestuia: aplicația web. Pentru implementarea acestui nucleu s-a utilizat framework-ul Flask, o alegere motivată de abordarea sa minimalistă și flexibilă.

Organizarea codului sursă este esențială pentru mentenabilitate. Proiectul adoptă o structură clară, conform principiului separării responsabilităților, așa cum este ilustrat în Figura 3.2. Fișierul `app.py` reprezintă punctul de intrare și nucleul logic al aplicației; aici se inițializează instanța Flask, se configurează conexiunea la baza de date, se definesc rutele, funcțiile de suport și logica de business. Directorul `templates/` conține toate fișierele HTML care alcătuiesc interfața vizuală, realizându-se astfel o separare eficientă a logicii de prezentare.

```

Studiu și implementarea soluțiilor cloud scalabile pentru aplicații web

|-- static/           # Director pentru fișiere statice (CSS, JS, imagini).
|-- templates/       # Director pentru template-urile HTML (interfața grafică).
|-- app.py           # Fișierul principal al aplicației, conține logica centrală.
|-- iis_health.html  # Pagină simplă pentru monitorizarea stării serverului IIS.

```

Figura 3.2: Structura ierarhică a proiectului Flask

Funcționalitatea aplicației este susținută de un set de biblioteci Python, fiecare având un rol bine definit, așa cum este evidențiat în Figura 3.3.

```

from flask import Flask, render_template, request, redirect, session, url_for, flash, jsonify
from mysql.connector import pooling, Error as MySQLError
import bcrypt, os, time
from datetime import datetime, date, timedelta
from collections import defaultdict
import itertools

```

Figura 3.3: Biblioteci principale utilizate în proiect

Rolul fiecărei biblioteci principale este detaliat mai jos:

- **flask:** Aceasta este biblioteca fundamentală. Din ea se importă obiecte esențiale:
 - *Flask:* Clasa principală utilizată pentru crearea instanței aplicației.
 - *render_template:* Funcția cheie pentru a invoca motorul de șabloane Jinja2 și a genera pagini HTML dinamice.

- *request*: Un obiect global care încapsulează toate informațiile despre cererea HTTP curentă (metodă, date de formular, parametri URL, antete).
 - *redirect, url_for*: Unelte pentru a gestiona fluxul de navigare, permițând redirectionarea utilizatorului către alte rute.
 - *session*: Un obiect de tip dicționar utilizat pentru a stoca date specifice unui utilizator între cereri (ex., ID-ul utilizatorului după logare), securizat printr-un cookie semnat criptografic.
 - *flash*: Un mecanism pentru a afișa mesaje temporare utilizatorului (ex., "Înregistrare reușită!").
 - *jsonify*: O funcție utilizată pentru a crea răspunsuri HTTP cu formatul application/json.
- **mysql.connector**: Biblioteca oficială pentru conectarea la MySQL. S-a utilizat sub-modulul *pooling* pentru a implementa *Connection Pooling*, o tehnică de optimizare ce menține un set de conexiuni deschise și refolosibile.
 - **bcrypt**: O bibliotecă esențială pentru securitate, dedicată procesării (hashing) parolilor. Utilizează algoritmul Blowfish și mecanismul de "salting" pentru a proteja împotriva atacurilor de tip forță brută și "rainbow table".
 - **Biblioteci Standard Python**:
 - *datetime, date, timedelta*: Folosite intensiv pentru a manipula și calcula intervale de timp, necesare pentru gestionarea perioadelor de rezervare.
 - *collections.defaultdict*: O structură de date avansată utilizată în algoritmul de calcul al zilelor blocate.
 - *itertools*: O bibliotecă puternică pentru crearea de iteratori eficienți, utilizată în motorul de recomandări.

Configurarea critică a instanței aplicației include definirea `app.secret_key`. Această cheie secretă este utilizată de Flask pentru a semna criptografic cookie-urile de sesiune, un mecanism vital pentru a preveni manipularea datelor de sesiune de către client și pentru a securiza sistemul de autentificare.

3.2 Fluxul 1 (End-to-End): Înregistrare și Autentificare Utilizator

Primul și cel mai fundamental flux de interacțiune cu platforma este cel prin care un vizitator devine un utilizator înregistrat și autentificat. Acest parcurs, cunoscut în terminologia de specialitate ca *user onboarding*, este esențial nu doar pentru personalizarea experienței, ci și pentru stabilirea unui perimetru de securitate în jurul funcționalităților tranzacționale, precum efectuarea de rezervări. Analiza acestui flux end-to-end, prezentată în continuare, va acoperi interfața vizuală, logica de procesare din backend și mecanismele de securitate implementate.

3.2.1 Interfața Frontend și Principii de Design (UI/UX)

Poarta de intrare în ecosistemul aplicației este reprezentată de formularele de înregistrare și autentificare. Proiectarea acestora nu este un aspect trivial, ci o decizie conștientă care influențează direct rata de conversie a vizitatorilor în utilizatori activi. Abordarea adoptată a fost una minimalistă, focusată pe eliminarea oricărei fricțiuni și pe ghidarea intuitivă a utilizatorului către finalizarea acțiunii.

3.2.1.1 Formularul de Înregistrare: Primul Contact cu Platforma

Punctul de intrare pentru un nou membru este materializat de pagina de înregistrare, servită de șablonul `register.html`. Această componentă de interfață a fost proiectată sub principiul ****colectării progresive a datelor**** (*progressive disclosure*). În loc să coplesim utilizatorul cu un formular extins, am solicitat doar setul minim de atribute indispensabile pentru crearea unui cont funcțional, așa cum se poate observa în Figura 3.4.

Fiecare câmp a fost selectat cu un scop precis:

- **Numele de utilizator:** Servește ca identitate publică în cadrul platformei, de exemplu, în secțiunea de recenzii.
- **Adresa de e-mail:** Acționează ca identificator unic, primar, în sistem și principalul canal de comunicare pentru notificări tranzacționale (confirmări de rezervare, etc.).
- **Parola:** Reprezintă cheia de acces, al cărei tratament criptografic în backend este un pilon central al securității aplicației.

Validarea la nivel de client, implementată prin atribute HTML5 native precum `required` și `type="email"`, constituie prima barieră de integritate a datelor. Această tehnică oferă utilizatorului un feedback imediat, în timp real, asupra corectitudinii informațiilor introduse, reducând numărul de cereri invalide care ajung la server și, implicit, optimizând utilizarea resurselor și fluidizând experiența de înregistrare.

Figura 3.4: Interfața formularului de înregistrare, proiectată pentru simplitate și claritate.

3.2.1.2 Formularul de Autentificare: Calea de Acces Rapid

Odată ciclul de înregistrare finalizat, parcursul de revenire al utilizatorului este deservit de pagina de autentificare (`login.html`). Design-ul acesteia, prezentat în Figura 3.5, este deliberat și mai minimalist, respectând principiul UX al ****eliminării distragerilor****. Pagina are un singur scop, o singură "chemare la acțiune" (*Call to Action*): autentificarea.

Prin solicitarea exclusivă a celor două credențiale – e-mail și parolă – se reduce sarcina cognitivă a utilizatorului și se accelerează la maximum procesul de accesare a contului. Acest formular acționează ca o poartă securizată; la submitere, acesta inițiază un dialog cu backend-ul, unde se va efectua validarea criptografică a parolei, un mecanism ce va fi disecat în secțiunea următoare.

Figura 3.5: Interfața formularului de autentificare, focalizată pe funcționalitate.

3.2.2 Procesarea Backend și Mecanismele de Securitate

În momentul în care un utilizator trimite un formular de înregistrare sau de autentificare, datele părăsesc browser-ul și ajung la serverul nostru. Aici, aplicația Flask preia controlul și inițiază un proces în mai mulți pași pentru a valida, securiza și persista aceste informații. Vom analiza fiecare mecanism tehnic implicat în acest flux.

3.2.2.1 Mecanismul 1: Validarea Datelor și Prevenirea Duplicatelor

Primul pas pe care serverul îl execută este validarea la nivel de server. Chiar dacă există validări în frontend, acestea pot fi ocolite. Prin urmare, validarea la nivel de server este esențială. Pentru ruta `/register`, aceasta înseamnă verificarea faptului că adresa de e-mail furnizată nu există deja în sistem.

Acest lucru se realizează printr-o interogare `SELECT` asupra tabeli `users`. Dacă interogarea returnează un rezultat, înseamnă că e-mailul este deja folosit, iar procesul de înregistrare este oprit. Utilizatorului i se afișează un mesaj de eroare prin mecanismul `flash` din Flask, fără a se crea un cont duplicat. Acest pas garantează integritatea și unicitatea datelor din baza de date.

```

1 # Fragment de cod din app.py: Verificarea existentei contului
2 cursor = db_connection_pool.get_connection().cursor()
3 cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
4 account = cursor.fetchone()
5
6 if account:
7     flash('Contul exista deja!', 'danger')
8     return redirect(url_for('register'))
9 # ... continua cu crearea contului daca nu exista

```

Listing 3.1: Interogare SQL pentru prevenirea conturilor duplicate.

3.2.2.2 Mecanismul 2: Securizarea Criptografică a Parolelor (bcrypt)

Cel mai critic aspect de securitate în gestiunea utilizatorilor este protejarea parolelor. Stocarea parolelor în clar este o practică inacceptabilă. Platforma noastră utilizează biblioteca `bcrypt` pentru a transforma parolele într-un format securizat, non-reversibil (hash).

La înregistrare, procesul este următorul:

1. Se ia parola introdusă de utilizator.
2. Se generează o "sare" (*salt*) – un șir de caractere aleatoriu, unic pentru fiecare utilizator.
3. Parola și sarea sunt combinate și trecute printr-un algoritm de hashing lent (Blowfish).

Rezultatul ('hashed_password') este cel care se stochează în baza de date. Deoarece sarea este unică, chiar și aceeași parolă va genera hash-uri diferite pentru utilizatori diferiți, anihilând eficiența atacurilor de tip *rainbow table*.

La autentificare, procesul este invers, dar la fel de sigur:

1. Se preia hash-ul stocat în baza de date pentru utilizatorul respectiv.
2. Se utilizează funcția `bcrypt.checkpw()`, care extrage automat sarea din hash-ul stocat.
3. Aplică același proces de hashing parolei introduse acum de utilizator, folosind sarea extrasă.
4. Compară cele două hash-uri. Doar dacă sunt identice, parola este corectă.

Astfel, parola în clar nu este niciodată manipulată sau comparată direct după înregistrarea inițială.

```

1 # La inregistrare:
2 parola_clar = request.form['password'].encode('utf-8')
3 parola_hashed = bcrypt.hashpw(parola_clar, bcrypt.gensalt())
4 cursor.execute("INSERT INTO users (password_hash) VALUES (%s)", (parola_hashed,))
5
6 # La autentificare:
7 parola_formular = request.form['password'].encode('utf-8')
8 parola_hashed_din_bd = user['password_hash'].encode('utf-8')
9 if bcrypt.checkpw(parola_formular, parola_hashed_din_bd):
10 # Parolele se potrivesc - autentificare reusita

```

Listing 3.2: Utilizarea bcrypt pentru hasharea și verificarea parolelor.

3.2.2.3 Mecanismul 3: Gestiunea Stării prin Sesiuni Securizate

Protocolul HTTP este, prin natura sa, "fără stare" (*stateless*). Serverul nu își amintește de la o cerere la alta cine este utilizatorul. Pentru a rezolva această problemă, folosim sistemul de sesiuni al Flask.

După o autentificare reușită, creăm o "sesiune" pentru utilizator. Acest lucru înseamnă că stocăm în obiectul special `session` din Flask informații de identificare, de obicei ID-ul unic al utilizatorului: `session['user_id'] = user['id']`.

Flask împachetează aceste date, le **semnează criptografic** folosind cheia secretă a aplicației (`app.secret_key`) și le trimite browser-ului sub forma unui cookie. La fiecare cerere viitoare, browser-ul va trimite acest cookie înapoi. Serverul Flask verifică semnătura. Dacă semnătura este validă, serverul are încredere în datele din cookie și știe cine este utilizatorul. Dacă cineva încearcă să modifice cookie-ul (de exemplu, să schimbe 'user_id'), semnătura nu se va mai potrivi, iar sesiunea va fi invalidată automat. Acesta este un mecanism robust care asigură persistența stării de autentificare într-un mod sigur.

```

# La autentificare reușită, se populează sesiunea
session['logged_in'] = True
session['user_id'] = user['id']
session['username'] = user['username']

```

```
# Într-o altă rută, se accesează datele din sesiune
if 'loggedin' in session:
    current_user_id = session['user_id']
    # ... folosește ID-ul pentru a face operațiuni personalizate
```

Figura 3.6: Crearea și utilizarea unei sesiuni de utilizator în Flask.

3.2.3 Procesarea Backend și Mecanismele de Afișare

În spatele interfeței vizuale stă un mecanism complex în backend, responsabil pentru interpretarea criteriilor utilizatorului, interogarea bazei de date și construirea dinamică a listei de rezultate relevante.

3.2.3.1 Mecanismul de Căutare și Filtrare Avansată

La trimiterea formularului de căutare, browser-ul execută o cerere HTTP POST către ruta `/search_results`. Spre deosebire de o căutare simplă, logica implementată în backend execută un proces de filtrare în doi pași, mult mai precis și mai eficient:

1. **Filtrarea Inițială a Proprietăților:** Într-o primă fază, se execută o interogare generală pe tabela `properties` pentru a selecta un set de "proprietăți candidate". Această interogare filtrează proprietățile pe baza locației introduse de utilizator și, important, exclude proprietățile care aparțin utilizatorului autentificat în acel moment (pentru a nu-și putea rezerva propria proprietate).
2. **Validarea Detaliată a Disponibilității Camerelor:** În a doua fază, aplicația iterează prin fiecare proprietate candidată și execută o a doua interogare, mult mai complexă, pe tabela `rooms`. Această interogare este nucleul tehnic al motorului de căutare: ea selectează doar camerele care sunt disponibile în intervalul de date specificat de utilizator.

Mecanismul de validare a disponibilității este implementat printr-o sub-interogare `NOT IN`. Se selectează toate camerele unei proprietăți (`'idrooms'`) care **nu se află** în lista de camere deja rezervate (`'reservations.room_id'`) în intervalul de timp care se suprapune cu cel dorit. Logica de suprapunere a intervalelor, `NOT (end_date <= start_date_cautare OR start_date >= end_date_cautare)`, este esențială pentru a garanta corectitudinea rezultatelor.

Fragmentul de cod din Figura 3.3 ilustrează această logică complexă de interogare.

Pas 1: Filtrarea proprietăților candidate

```
params = []
if destination:
    base_query += " AND (name LIKE %s OR city LIKE %s)"; params.extend([f"%{destination}%", f"%{destination}%"])
```

Figura 3.7: Interfața formularului de autentificare, focusată pe funcționalitate.

```
cursor.execute(base_query, tuple(params))
candidate_properties = cursor.fetchall()
```

Figura 3.8: Interfața formularului de autentificare, focusată pe funcționalitate.

```

1 properties_that_match = []
2 for prop in candidate_properties:
3     # Pas 2: Verificarea disp camerelor pentru fiecare proprietate
4     room_query = """
5         SELECT name, capacity, price, COUNT(*) as count,
6             GROUP_CONCAT(idrooms) as room_ids
7         FROM rooms
8         WHERE property_id = %s AND available = TRUE AND idrooms NOT IN (
9             SELECT room_id FROM reservations
10            WHERE status = 'confirmed' AND NOT (end_date <= %s OR start_date >= %s)
11        )
12        GROUP BY name, capacity, price
13    """
14    cursor.execute(room_query, (prop['property_id'], start_date, end_date))
15    available_room_groups = cursor.fetchall()
16
17    # Pas 3 (Optional): Invocarea motorului de recomandari
18    if available_room_groups:
19        recommendation = get_recommendations(available_room_groups, adults, rooms_needed)
20        if recommendation:
21            properties_that_match.append(prop)
22
23 return render_template("search_results.html", properties=properties_that_match, ...)

```

Listing 3.3: Logica de filtrare în doi pași și interogarea pentru validarea disponibilității camerelor.

3.2.3.2 Motorul de Recomandări și Randarea Rezultatelor

După identificarea grupurilor de camere disponibile, logica nu se oprește la o simplă afișare. Se invocă o funcție specializată, `get_recommendations()`, care are rolul de a găsi combinația optimă de camere pentru a satisface cerințele utilizatorului (număr de adulți și număr de camere dorite). Această abordare transformă o simplă căutare într-un sistem inteligent de asistență în planificarea călătoriei.

Doar proprietățile pentru care se găsește o recomandare validă sunt trimise către șablonul `search_results.html`, unde sunt randate dinamic folosind motorul Jinja2, așa cum a fost descris anterior. Rezultatul final este o listă de opțiuni nu doar disponibile, ci și perfect potrivite pentru nevoile utilizatorului, așa cum se poate vedea în Figura 3.9.

```

params = []
if destination:
    base_query += " AND (name LIKE %s OR city LIKE %s)"; params.extend([f"%{destination}%", f"%{destination}%"])

```

Figura 3.9: Exemplu de pagină cu rezultatele căutării, afișând proprietățile disponibile.

Gestiunea Rutelor și a Ciclului Cerere-Răspuns HTTP

Flask gestionează cererile HTTP printr-un sistem de rutare bazat pe decoratori. Decoratorul `@app.route()` leagă un URL specific de o funcție Python care va fi executată atunci când acel URL este accesat.

3.2.4 Ruta (pagina principală)

Pagina principală a unei aplicații web reprezintă prima impresie a unui utilizator atunci când o accesează. Aceasta prezintă o imagine de ansamblu a platformei, afișând o selecție de proprietăți disponibile începând cu ziua curentă accesării și oferind posibilitatea de a căuta în funcție de mai multe criterii. Pentru utilizatorii autentificați, pagina principală exclude din a prezenta propriile cazări înregistrare.

Metoda HTTP folosită în implementarea rutei este GET deoarece ruta este destinată exclusiv vizualizărilor de informații, nefiind implicate trimitere de date de la client la server.

Mod de funcționare:

Aplicația a fost structurată în șapte module, fiecare modul având un rol bine definit în arhitectura generală. Acestea sunt următoarele:

1. Modulul de aplicație web în Python Flask este prezent pe stația de lucru și include rute importante precum /, /login, /register, /logout, /profile, /admin, /reserve pentru gestionarea logicii unui sistem de management al rezervărilor, autentificarea utilizatorilor și interacțiunea cu baza de date. Funcționalitățile rutelor sunt:
 - /login - autentificarea utilizatorilor și validarea credențialelor.
 - /register - înregistrarea utilizatorilor noi și validarea datelor.
 - /logout - finalizarea sesiunii active.
 - /reserve - rezervarea unui hotel (operațiune de tip CRUD).
 - /profile - afișarea și actualizarea profilelor utilizatorilor.
 - /admin - rută pentru utilizatorii cu rol de administrator.
 - / - pagina de acasă cu o interfață generală sau redirecționare în funcție de sesiune.
2. Modulul de load balancing (Nginx): Nginx (instalat pe stația de lucru) se ocupă de distribuirea cererilor primite de la utilizatori către cele două servere: aplicația locală Flask și serverul IIS de pe VM2, ceea ce îmbunătățește scalabilitatea soluției implementate. Load balancer-ul are mai multe roluri cum ar fi:
 - Reverse proxy care asigură redirecționarea traficului.
 - Verificarea și detectarea automată a sănătății server-ului și scoaterea temporară din funcționare până când sunt rezolvate erorile.
 - Balansarea cererilor trimise către servere, astfel încât cereri sunt preluate secvențial.
 - Măsură de rezervă în cazul în care unul dintre servere devine inaccesibil, traficul va fi redirecționat către următorul server disponibil.

Bibliografie

- [1] Adam Hayes. Scalability: What a scalable company is and examples. <https://www.investopedia.com/terms/s/scalability.asp>. Accesat: 08 mai 2025.
- [2] Kirill Stashevsky Nadejda Alkhaldi. What is software scalability, and why should your company take it seriously? <https://itrexgroup.com/blog/what-is-software-scalability>. Accesat: 08 mai 2025.
- [3] LinkedIn Computer Engineering. What is the best way to ensure hardware scalability in a computer system? <https://www.linkedin.com/advice/1/what-best-way-ensure-hardware-scalability-2pf1e>. Accesat: 08 mai 2025.
- [4] Wikipedia. Bază de date distribuită. https://ro.wikipedia.org/wiki/Baz%C4%83_de_date_distribuit%C4%83. Accesat: 08 mai 2025.
- [5] Yuliia Podorozhko. Scalability in software development: A 101 guide for businesses. <https://madappgang.com/blog/scalability-in-software-development/#:~:text=Scalability%20in%20software%20development%20refers,users%2C%20customers%2C%20or%20requests>. Accesat: 09 mai 2025.
- [6] Silviu Stroe. Ce este o aplicație web? <https://brainic.io/ro/blog/ce-este-o-aplicatie-web/>. Accesat: 09 mai 2025.
- [7] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP>. Accesat: 09 mai 2025.
- [8] Developer Mozilla. Http documentation. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/User-Agent>. Accesat: 09 mai 2025.
- [9] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server. Accesat: 09 mai 2025.
- [10] Developer Mozilla. Http documentation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL. Accesat: 09 mai 2025.
- [11] Tutorialspoint. Http - requests. https://www.tutorialspoint.com/http/http_requests.htm. Accesat: 10 mai 2025.
- [12] Tutorialspoint. Http - responses. https://www.tutorialspoint.com/http/http_responses.htm. Accesat: 12 mai 2025.
- [13] Flask Tutorial. Geeksforgeeks. <https://www.geeksforgeeks.org/flask-tutorial/>. Accesat: 12 mai 2025.
- [14] Flask Documentation. Flask. <https://flask.palletsprojects.com/en/stable/>. Accesat: 12 mai 2025.

- [15] Wikipedia. Flask (web framework). [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework)). Accesat: 12 mai 2025.
- [16] Abraham Silberschatz; Henry Korth; S. Sudarshan. *Database System Concepts*. McGraw Hill, 2010.

Anexa A

Rezolvare ”Cinci Pâini”

A aduce 3 pâini, iar B aduce 2 pâini. C plătește 5 lei. Se împarte fiecare pâine în 3 părți egale: $3 \times 3 + 2 \times 3 = 15$ bucăți. Fiecare mesean capătă câte 5 bucăți. Reiese că A i-a dat lui C $3 \times 3 - 5 = 4$ pâini, iar B i-a dat $2 \times 3 - 5 = 1$ pâine. Împărțeală cinstită este deci 4 lei pentru A, și 1 leu pentru B.

Anexa B

Alte povești și basme

Vezi tabela ?? cu alte povești și autorii/culegătorii lor.

```

1 function [ out ] = Cod( varinput )
2 % acesta este un comentariu
3 % param intrare varinput
4 % param iesire out
5
6 rez = 0;
7 n=10;
8 for i=1:n
9     for j=i+1:n-1
10        if abs(-2)<abs(-3)
11            rez = rez +1;
12        end
13    end
14 end
15 end

```

-	Dănilă prepeleac	Prâslea cel voi- nic și merele de aur	Făt-Frumos din lacrimă	Greuceanu
	Ion Creangă	Petre Ispirescu	Mihai Eminescu	Petre Ispirescu

Tabela B.1: Alte basme.